

E-Commerce Application – DevOps & Microservices Certification Project

Project Overview

Business Background

ShopSphere is a global E-Commerce platform headquartered in Germany that provides online shopping services for millions of users across multiple countries. The company offers products and services across categories such as:

- Electronics
- Fashion and Apparel
- Grocery and Essentials
- Home Appliances
- Furniture
- Beauty and Personal Care
- Digital Products
- Subscription Services

Initially, ShopSphere used a monolithic application architecture to manage all business operations including product management, customer management, order processing, payments, inventory, and shipment tracking.

As the business expanded rapidly, the company started facing challenges in:

- Managing large application deployments
- Scaling individual modules independently
- Increasing infrastructure maintenance overhead
- Delayed software releases
- Frequent deployment failures
- Long testing cycles
- Performance bottlenecks during high traffic seasons
- Manual infrastructure provisioning
- Lack of centralized monitoring and observability

To overcome these limitations, ShopSphere decided to modernize its application platform by adopting:

- Microservices Architecture
- DevOps Practices
- CI/CD Automation

- Containerization
- Kubernetes-based Deployments
- Infrastructure as Code (IaC)
- Cloud-Native Monitoring Solutions

The company selected Amazon Web Services (AWS) as the primary cloud provider for infrastructure, databases, application hosting, monitoring, and deployment automation.

Business Objectives

The primary goal of ShopSphere is to:

- Deliver product updates frequently and reliably
 - Reduce deployment failures
 - Improve application scalability
 - Automate infrastructure provisioning
 - Automate testing and deployments
 - Improve monitoring and alerting capabilities
 - Enable rapid feedback between developers and testers
 - Reduce manual operational effort
 - Improve system reliability and availability
-

Current Challenges

Due to the existing monolithic architecture and manual operational processes, ShopSphere is currently facing the following issues:

Application Challenges

- Building and deploying the monolithic application is complex
- Application startup time is high
- Large codebase is difficult to maintain
- Dependency management is difficult
- Small changes require full application deployment
- Performance degradation during high traffic
- Difficult to isolate failures

Testing Challenges

- Manual testing consumes significant time

- Regression testing is difficult
- Delayed bug identification
- Lack of automated test execution

Deployment Challenges

- Manual deployment processes are error-prone
- Incremental builds are difficult to manage
- Rollback process is complicated
- Downtime during deployments

Infrastructure Challenges

- Infrastructure provisioning is manual and time-consuming
- Environment inconsistency across DEV/UAT/PROD
- Scaling infrastructure manually is difficult
- Lack of Infrastructure as Code practices

Monitoring Challenges

- No centralized monitoring system
 - Manual application monitoring
 - Lack of real-time alerting
 - Difficult root cause analysis
-

Proposed Solution

ShopSphere plans to implement a cloud-native microservices platform using DevOps best practices.

The solution will include:

- Spring Boot Microservices
 - REST APIs
 - AWS Cloud Infrastructure
 - Docker Containerization
 - Kubernetes Orchestration
 - Jenkins CI/CD Pipelines
 - Terraform Infrastructure Automation
 - Ansible Configuration Management
 - Prometheus Monitoring
 - Grafana Dashboards
-

Microservices Architecture

The E-Commerce platform will be divided into multiple independent microservices.

Core Microservices

1. User Service

Responsible for:

- User registration
- User login
- Profile management
- Password reset
- Role management

2. Product Service

Responsible for:

- Product catalog
- Product details
- Category management
- Product search
- Product filtering
- Product inventory

3. Cart Service

Responsible for:

- Add to cart
- Remove from cart
- Update cart quantity
- Cart summary

4. Order Service

Responsible for:

- Order placement
- Order history
- Order status tracking
- Order cancellation

5. Payment Service

Responsible for:

- Payment processing
- Payment validation
- Payment status
- Refund handling

6. Shipping Service

Responsible for:

- Shipment tracking
- Delivery status
- Address validation
- Logistics integration

7. Notification Service

Responsible for:

- Email notifications
 - SMS notifications
 - Order alerts
 - Promotional notifications
-

Non-Functional Requirements

Scalability

- Individual services should scale independently
- Kubernetes Horizontal Pod Autoscaling should be supported

High Availability

- Multiple replicas of microservices
- Load balancing enabled
- Fault-tolerant architecture

Security

- JWT Authentication
- HTTPS Communication
- Secure API access

- Role-based authorization

Performance

- Response time should be less than 2 seconds
- Optimized database queries
- Caching support

Reliability

- Automated backups
 - Retry mechanisms
 - Health checks
 - Self-healing deployments
-

Technology Stack

Component	Technology
Programming Language	Java 17
Framework	Spring Boot
Build Tool	Maven
Database	MySQL / H2
Version Control	Git
Repository	GitHub
CI/CD	Jenkins
Containerization	Docker
Orchestration	Kubernetes
Configuration Management	Ansible
Infrastructure Provisioning	Terraform
Monitoring	Prometheus
Dashboard Visualization	Grafana
Cloud Provider	AWS

DevOps Workflow

Step 1 – Developer Activity

- Developer writes code
- Code pushed to GitHub repository

- Pull request created
 - Code review performed
-

Step 2 – Continuous Integration

Jenkins automatically performs:

- Source code checkout
 - Maven build
 - Unit testing
 - Code quality validation
 - Artifact generation
-

Step 3 – Automated Testing

Testing includes:

- JUnit test execution
- TestNG execution
- API testing
- Integration testing

HTML reports are generated automatically.

Step 4 – Docker Image Creation

After successful build:

- Docker image is created
 - Image tagged with build version
 - Image pushed to Docker Registry
-

Step 5 – Infrastructure Provisioning

Terraform provisions:

- EC2 Instances
- VPC
- Security Groups
- RDS Database

- Kubernetes Cluster
 - Monitoring Servers
-

Step 6 – Configuration Management

Ansible automatically configures:

- Java
 - Docker
 - Kubernetes tools
 - Jenkins agents
 - Application dependencies
-

Step 7 – Kubernetes Deployment

Application deployment includes:

- Deployment YAMLs
 - Services
-

Step 8 – Monitoring and Observability

Prometheus monitors:

- CPU utilization
- Memory usage
- Disk usage
- Pod health
- Application uptime
- Request latency

Grafana dashboards visualize metrics in real time.

CI/CD Pipeline Flow

```
Developer Pushes Code → GitHub → Jenkins Pipeline Triggered
↓
Code Checkout
↓
Maven Build
↓
```


JUnit/TestNG Execution
↓
Generate Reports
↓
Docker Image Build
↓
Push Docker Image
↓
Terraform Infrastructure Provisioning
↓
Ansible Configuration
↓
Deploy to Kubernetes
↓
Automated Testing
↓
Deploy to Production
↓
Monitoring using Prometheus & Grafana

Jenkins Pipeline Stages

Suggested Pipeline Stages

1. Checkout Source Code
 2. Build Application
 3. Run Unit Tests
 4. Static Code Analysis
 5. Generate Build Artifact
 6. Build Docker Image
 7. Push Docker Image
 8. Provision Infrastructure
 9. Configure Servers
 10. Deploy to Kubernetes
 11. Smoke Testing
 12. Production Deployment
 13. Monitoring Validation
-

Terraform Requirements

Terraform should automate creation of:

- VPC
 - Subnets
 - Internet Gateway
 - Route Tables
 - EC2 Instances
 - Security Groups
 - RDS Database
 - EKS Cluster
 - S3 Buckets
-

Ansible Requirements

Ansible Playbooks should automate:

- Java installation
 - Docker installation
 - Kubernetes tools setup
 - Application configuration
 - User management
 - File deployment
 - Service startup
-

Monitoring Requirements

Prometheus Metrics

Monitor the following:

- CPU Utilization
 - Memory Usage
 - Disk Space Usage
 - JVM Heap Memory
 - API Response Time
 - HTTP Request Count
 - Pod Restart Count
 - Database Health
-

Grafana Dashboards

Create dashboards for:

- Infrastructure Monitoring
 - Application Monitoring
 - Kubernetes Monitoring
 - Database Monitoring
 - Business Metrics
-

AWS Architecture

AWS Services Used

AWS Service	Purpose
EC2	Virtual Servers
RDS	Managed Database
EKS	Kubernetes Cluster
S3	Artifact Storage
IAM	Access Management
CloudWatch	Cloud Monitoring
Route53	DNS Management
ALB	Load Balancer

Deployment Environments

Development Environment (DEV)

Purpose:

- Developer testing
- Feature validation

Testing Environment (UAT)

Purpose:

- QA testing
- Integration testing
- Automation testing

Production Environment (PROD)

Purpose:

- Live customer traffic
 - Highly available deployment
-

Security Best Practices

- Enable HTTPS
 - Use encrypted secrets
 - Implement RBAC in Kubernetes
 - Use IAM roles with least privilege
 - Enable vulnerability scanning
 - Secure Docker images
 - Rotate credentials regularly
-

Expected Deliverables

Application Deliverables

- Spring Boot microservices
- REST APIs
- Database scripts
- Unit test cases
- HTML test reports

DevOps Deliverables

- GitHub repository
 - Jenkins pipeline
 - Docker images
 - Kubernetes manifests
 - Terraform scripts
 - Ansible playbooks
 - Monitoring dashboards
-

Expected Outcome

After implementation, ShopSphere should achieve:

- Faster release cycles
 - Automated deployments
 - Reduced manual effort
 - Improved scalability
 - High availability
 - Better monitoring visibility
 - Reliable infrastructure provisioning
 - Faster issue detection
 - Improved software quality
-

Conclusion

The E-Commerce DevOps Microservices Project demonstrates a complete enterprise-grade cloud-native application lifecycle implementation using modern DevOps and Microservices practices.

The project helps teams understand:

- Real-world CI/CD workflows
- Cloud infrastructure automation
- Container orchestration
- Microservices architecture
- Monitoring and observability
- DevOps automation tools
- Production-grade deployment strategies